HTTP Observatory Report

Report Feedback

Scan summary: ambr3.github.io/weather/

B

Score: 75 / 100
Scan Time: Just now
Tests Passed: 10 / 12

Scan results

Scoring

Test	Score	Reason	Recommendation
Content Security Policy (CSP)	0* ✓	Content Security Policy (CSP) implemented without **'unsafe-inline'** or **'unsafe-eval'**	Set **default-src** to **'none'**
Cookies	-	No cookies detected	None
Cross Origin Resource Sharing (CORS)	0 ✓	Content is not visible via cross-origin resource sharing (CORS) files or headers.	None

Test	Score	Reason	Recommendation
Redirection	0 ✓	Initial redirection is to HTTPS on same host, final destination is HTTPS	None
Referrer Policy	0* ✓	**Referrer-Policy** header set to **no-referrer**, **same-origin**, **strict-origin** or **strict-origin-when-cross-origin**.	None
Strict Transport Security (HSTS)	0 ✓	**Strict-Transport-Security** header set to a minimum of six months (15768000).	Consider preloading: this requires adding the **preload** and **includeSubDomains** directives and setting **max-age** to at least **31536000** (1 year), and submitting your site to https://hstspreload.org/.
Subresource Integrity	-	Subresource Integrity (SRI) not implemented, but all scripts are loaded from a similar origin.	Add SRI for bonus points.
X-Content-Type-Options	−5 ✗	**X-Content-Type-Options** header not implemented.	Set to **nosniff**.
X-Frame-Options	−20 ✗	**X-Frame-Options** (XFO) header not implemented.	Implement frame-ancestors CSP.
Cross Origin Embedder Policy	0 ✓	**Cross-Origin-Embedder-Policy** header not implemented.	Set to **require-corp** or **credentialless**.
Cross Origin Opener Policy	0 ✓	**Cross-Origin-Opener-Policy** header not implemented.	Set to **same-origin**, **same-origin-allow-popups** or **noopener-allow-popups**.

Test	Score	Reason	Recommendation
Cross Origin Resource Policy	-	Cross Origin Resource Policy (CORP) is not implemented (defaults to **`cross-origin`**).	None

* Normally awards bonus points, however, in this case they are not included in the overall score (find out why).

CSP analysis

✓ Content Security Policy (CSP) implemented without **`'unsafe-inline'`** or **`'unsafe-eval'`**

Test	Result	Info
Blocks execution of inline JavaScript by not allowing **`'unsafe-inline'`** inside **`script-src`**	✓	Blocking the execution of inline JavaScript provides CSP's strongest protection against cross-site scripting attacks. Moving JavaScript to external files can also help make your site more maintainable.
Blocks execution of JavaScript's **`eval()`** function by not allowing **`'unsafe-eval'`** inside **`script-src`**	✓	Blocking the use of JavaScript's **`eval()`** function can help prevent the execution of untrusted code.
Blocks execution of plug-ins, using **`object-src`** restrictions	✓	Blocking the execution of plug-ins via **`object-src 'none'`** as inherited from **`default-src`** can prevent attackers from loading Flash or Java in the context of your page.
Blocks inline styles by not allowing **`'unsafe-inline'`** inside **`style-src`**	✓	Blocking inline styles can help prevent attackers from modifying the contents or appearance of your page. Moving styles to external stylesheets can also help make your site more maintainable.
Blocks loading of active content over HTTP or FTP	✓	Loading JavaScript or plugins can allow a man-in-the-middl to execute arbitrary code or your website. Restricting your

Test	Result	Info
		policy and changing links to HTTPS can help prevent this.
Blocks loading of passive content over HTTP or FTP	✓	This site's Content Security Policy allows the loading of passive content such as images or videos over insecure protocols such as HTTP or FTP. Consider changing them to load them over HTTPS.
Clickjacking protection, using **`frame-ancestors`**	✗	The use of CSP's **`frame-ancestors`** directive offers fine-grained control over who can frame your site.
Deny by default, using **`default-src 'none'`**	✗	Denying by default using **`default-src 'none'`** can ensure that your Content Security Policy doesn't allow the loading o resources you didn't intend to allow.
Restricts use of the **<base>** tag by using **`base-uri 'none'`**, **`base-uri 'self'`**, or specific origins.	✓	The **<base>** tag can be used to trick your site into loading scripts from untrusted origins.
Restricts where **<form>** contents may be submitted by using **`form-action 'none'`**, **`form-action 'self'`**, or specific URIs	✓	Malicious JavaScript or content injection could modify wher sensitive form data is submitted to or create additional form for data exfiltration.
Uses CSP3's **`'strict-dynamic'`** directive to allow dynamic script loading (optional)	-	**`'strict-dynamic'`** lets you use a JavaScript shim loader t load all your site's JavaScript dynamically, without having to track **`script-src`** origins.

Cookies

No cookies detected

Raw server headers

Header	Value
Age	0
Via	1.1 varnish
Date	Mon, 02 Nov 2026 18:23:35 GMT
Etag	W/"6ae8c719-3fa3"
Vary	Accept-Encoding
Server	GitHub.com
Expires	Mon, 02 Nov 2026 18:33:35 GMT
X-Cache	MISS
X-Timer	S1793643815.122005,VS0,VE82
Connection	close
X-Served-By	cache-bfi-kbfi7400096-BFI
Content-Type	text/html; charset=utf-8
X-Cache-Hits	0
Accept-Ranges	bytes
Cache-Control	max-age=600
Last-Modified	Mon, 02 Nov 2026 17:23:37 GMT
X-Proxy-Cache	MISS
Content-Length	4249

Header	Value
X-Fastly-Request-Id	bca2544f339bd3d8ecc621610f1df075fd832d1b
X-Github-Request-Id	5DEA:3848ED:2A17A:4AB26:6AE8D527
X-Github-Edge-Region	westus2
Strict-Transport-Security	max-age=31556952
Access-Control-Allow-Origin	*

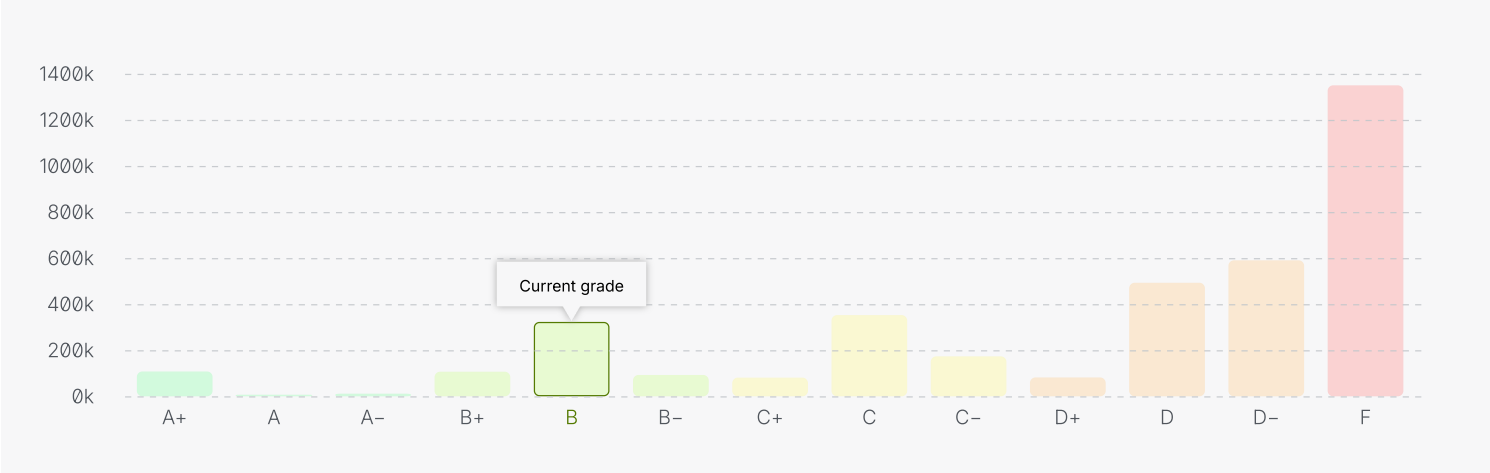
Scan history

Changes in score over time

Date	Score	Grade
2 Nov 2026, 18:23:35	75	B

Benchmark comparison

Performance trends from the past year



Refer to this graph to assess the website's current status. By following the recommendations provided and rescanning, you can expect an improvement in the website's grade.

mdn_

Your blueprint for a better internet.